

LAPORAN PRAKTIKUM PERTEMUAN 4
PEMROGRAMAN MOBILE



Dosen Pengampu:

Jefry Sunupurwa Asri, S.Kom., M.Kom.

Disusun Oleh:

Ramsay Abelson 20240801042

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS ESA UNGGUL
TANGERANG
2026

LAPORAN PRAKTIKUM PERTEMUAN 4

Mengambil Data dari REST API (HTTP, Async, dan FutureBuilder)

A. Tujuan Praktikum

1. Memahami pemrograman asinkron di Dart menggunakan Future, async, dan await.
2. Mengambil data dari REST API memakai paket http dan menguraikan JSON.
3. Membuat class model dengan konstruktor fromJson.
4. Menampilkan data asinkron dengan FutureBuilder lengkap dengan status loading, error, dan data.
5. Menangani kegagalan jaringan dan menyediakan tombol coba lagi.

B. Alat dan Bahan

1. Flutter SDK, Visual Studio Code (WSL Ubuntu), dan emulator Android.
2. Proyek Flutter dengan paket http.
3. Koneksi internet dan API publik JSONPlaceholder (jsonplaceholder.typicode.com).

C. Langkah Praktikum dan Hasil

1. Mengenal Future dan FutureBuilder (Bagian A)

Bagian A memperkenalkan Future tanpa memakai internet. Fungsi ambilSalam() mensimulasikan pengambilan data yang butuh 2 detik menggunakan Future.delayed, lalu hasilnya ditampilkan dengan FutureBuilder. Selama menunggu tampil CircularProgressIndicator, kemudian berganti menjadi teks "Halo dari masa depan!". Future dibuat di initState(), bukan di build(), agar tidak diulang setiap layar dibangun ulang.

Kode 1. main.dart (Bagian A)

```
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 4',
      theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
      home: const SalamPage(),
    );
  }
}
```

```

Future<String> ambilSalam() async {
  await Future.delayed(const Duration(seconds: 2));
  return 'Halo dari masa depan!';
}

class SalamPage extends StatefulWidget {
  const SalamPage({super.key});

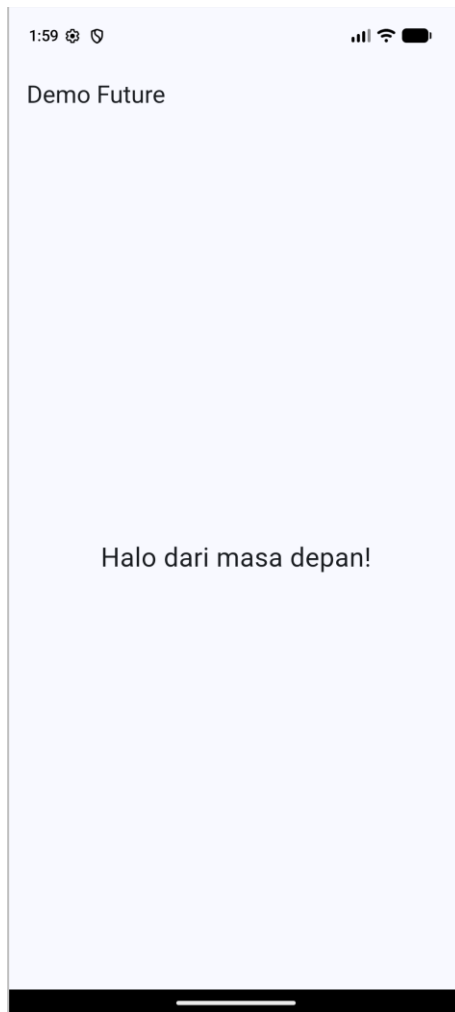
  @override
  State<SalamPage> createState() => _SalamPageState();
}

class _SalamPageState extends State<SalamPage> {
  late Future<String> _future;

  @override
  void initState() {
    super.initState();
    future = ambilSalam();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Demo Future')),
      body: Center(
        child: FutureBuilder<String>(
          future: _future,
          builder: (context, snapshot) {
            if (snapshot.connectionState == ConnectionState.waiting) {
              return const CircularProgressIndicator();
            }
            if (snapshot.hasError) {
              return Text('Error: ${snapshot.error}');
            }
            return Text(snapshot.data!, style: const TextStyle(fontSize: 24));
          },
        ),
      ),
    );
  }
}

```



Gambar 1. Tampilan Demo Future setelah 2 detik

2. Menyiapkan Paket HTTP (Bagian B)

Paket http ditambahkan ke proyek dengan perintah berikut di terminal pada folder proyek:

```
| flutter pub add http
```

Perintah ini mengunduh paket http beserta dependensinya dan mencatatnya di pubspec.yaml. Agar aplikasi release Android dapat mengakses internet, izin INTERNET perlu ditambahkan pada android/app/src/main/AndroidManifest.xml, tepat di atas tag <application (mode debug umumnya sudah memilikinya):

```
| <uses-permission android:name="android.permission.INTERNET"/>
```

```

→ mobile git:(master) X flutter pub add http
Woah! You appear to be trying to run flutter as root.
We strongly recommend running the flutter tool without superuser privileges.
/
🔗
Resolving dependencies...
Downloading packages... (1.0s)
  async 2.11.0 (2.13.1 available)
  boolean_selector 2.1.1 (2.1.2 available)
  characters 1.3.0 (1.4.1 available)
  clock 1.1.1 (1.1.3 available)
  collection 1.18.0 (1.19.1 available)
  cupertino_icons 1.0.8 (2.0.0 available)

```

Gambar 2. Menjalankan flutter pub add http

3. Model, Pengambil Data, FutureBuilder, dan Halaman Detail (Bagian C–E)

Bagian C, D, dan E digabung dalam satu berkas main.dart karena saling bergantung: halaman daftar (D) memakai model dan fungsi dari C, dan membuka halaman detail (E) saat sebuah item diketuk. Isi berkas main.dart adalah sebagai berikut.

Kode 2. main.dart (Bagian C, D, dan E)

```

import 'dart:convert';

import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 4',
      theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
      home: const PenggunaPage(),
    );
  }
}

// ===== Bagian C: Model dan fungsi pengambil data =====

class Pengguna {
  final int id;
  final String name;
  final String email;
  final String phone;
  final String website;

  const Pengguna({
    required this.id,
    required this.name,
    required this.email,
    required this.phone,
    required this.website,
  });

  factory Pengguna.fromJson(Map<String, dynamic> json) {
    return Pengguna(
      id: json['id'] as int,
      name: json['name'] as String,

```

```

        email: json['email'] as String,
        phone: json['phone'] as String,
        website: json['website'] as String,
    );
}
}

Future<List<Pengguna>> ambilPengguna() async {
    final uri = Uri.parse('https://jsonplaceholder.typicode.com/users');
    final response = await http.get(uri).timeout(const Duration(seconds: 10));
    if (response.statusCode != 200) {
        throw Exception('Gagal memuat data (kode ${response.statusCode})');
    }
    final List<dynamic> data = jsonDecode(response.body);
    return data
        .map((e) => Pengguna.fromJson(e as Map<String, dynamic>))
        .toList();
}

// ===== Bagian D: FutureBuilder =====

class PenggunaPage extends StatefulWidget {
    const PenggunaPage({super.key});

    @override
    State<PenggunaPage> createState() => _PenggunaPageState();
}

class _PenggunaPageState extends State<PenggunaPage> {
    late Future<List<Pengguna>> _future;

    @override
    void initState() {
        super.initState();
        _future = ambilPengguna();
    }

    void _muatUlang() {
        setState(() {
            _future = ambilPengguna();
        });
    }

    @override
    Widget build(BuildContext context) {
        return Scaffold(
            appBar: AppBar(
                title: const Text('Daftar Pengguna'),
                actions: [
                    IconButton(icon: const Icon(Icons.refresh), onPressed: _muatUlang),
                ],
            ),
            body: FutureBuilder<List<Pengguna>>(
                future: _future,
                builder: (context, snapshot) {
                    if (snapshot.connectionState == ConnectionState.waiting) {
                        return const Center(child: CircularProgressIndicator());
                    }
                    if (snapshot.hasError) {
                        return Center(
                            child: Padding(
                                padding: const EdgeInsets.all(24),
                                child: Column(
                                    mainAxisAlignment: MainAxisAlignment.min,
                                    children: [
                                        const Icon(Icons.error_outline, size: 48, color: Colors.red),
                                        const SizedBox(height: 8),
                                        Text(
                                            'Terjadi kesalahan:\n${snapshot.error}',
                                            textAlign: TextAlign.center,
                                        ),
                                        const SizedBox(height: 12),
                                    ],
                                ),
                            ),
                        );
                    }
                },
            ),
        );
    }
}

```

```

        ElevatedButton(
          onPressed: _muatUlang,
          child: const Text('Coba lagi'),
        ),
      ],
    ),
  ),
);
}
final data = snapshot.data!;
return ListView.builder(
  itemCount: data.length,
  itemBuilder: (context, i) {
    final p = data[i];
    return ListTile(
      leading: CircleAvatar(child: Text(p.name[0])),
      title: Text(p.name),
      subtitle: Text(p.email),
      trailing: const Icon(Icons.chevron_right),
      onTap: () {
        Navigator.push(
          context,
          MaterialPageRoute(
            builder: (_) => DetailPenggunaPage(pengguna: p),
          ),
        );
      },
    );
  },
);
},
);
},
);
};
}
}

// ===== Bagian E: Halaman detail =====

class DetailPenggunaPage extends StatelessWidget {
  final Pengguna pengguna;

  const DetailPenggunaPage({super.key, required this.pengguna});

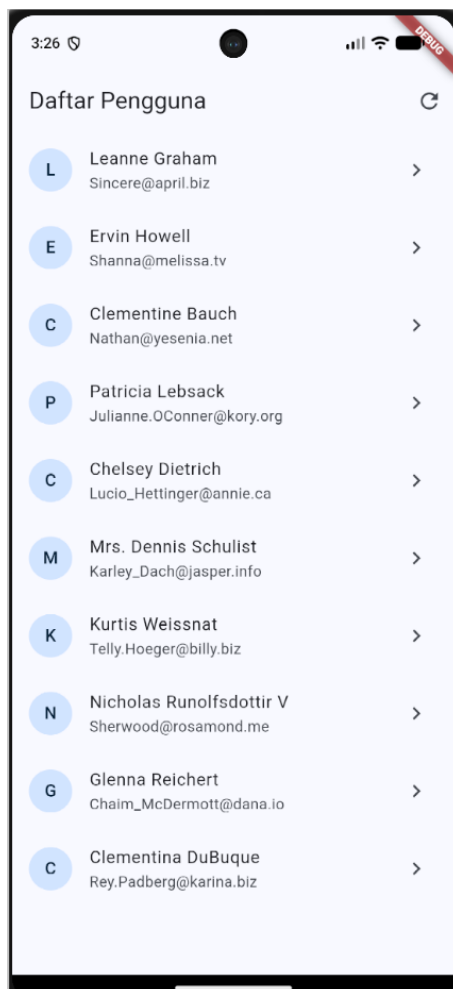
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(pengguna.name)),
      body: ListView(
        children: [
          ListTile(
            leading: const Icon(Icons.email),
            title: Text(pengguna.email),
          ),
          ListTile(
            leading: const Icon(Icons.phone),
            title: Text(pengguna.phone),
          ),
          ListTile(
            leading: const Icon(Icons.language),
            title: Text(pengguna.website),
          ),
        ],
      ),
    );
  }
}

```

Penjelasan singkat:

1. Bagian C: class Pengguna dengan factory fromJson mengubah Map JSON menjadi objek. Fungsi ambilPengguna() mengirim permintaan GET ke /users dengan batas waktu 10 detik, melempar Exception bila kode status bukan 200, lalu menguraikan JSON menjadi List<Pengguna>.
2. Bagian D: PenggunaPage membuat Future di initState() dan menampilkannya dengan FutureBuilder. Tiga keadaan ditangani: loading (CircularProgressIndicator), galat (ikon, pesan, dan tombol Coba lagi), serta data (ListView.builder). Tombol refresh di AppBar memanggil _muatUlang().
3. Bagian E: DetailPenggunaPage menampilkan email, telepon, dan situs web pengguna yang dipilih, dibuka lewat Navigator.push.

Hasilnya, sepuluh pengguna dari API tampil dalam daftar setelah indikator loading singkat.



Gambar 3. Tampilan Daftar Pengguna (Bagian D)

4. Latihan Mandiri

Latihan mandiri dikerjakan dengan mengubah kode Bagian C–E sebagai berikut:

1. Model Pengguna ditambah kolom username dan city (data bersarang json['address']['city']), keduanya ditampilkan di halaman detail.
2. ListView dibungkus RefreshIndicator sehingga daftar dapat ditarik ke bawah untuk memuat ulang.
3. Bila daftar kosong, ditampilkan teks "Tidak ada data".
4. Judul AppBar menampilkan jumlah pengguna setelah data berhasil dimuat, misalnya "Daftar Pengguna (10)".

Kode 3. main.dart untuk Latihan Mandiri

```
import 'dart:convert';

import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 4',
      theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
      home: const PenggunaPage(),
    );
  }
}

// Latihan 1: tambah username dan kota (data bersarang)
class Pengguna {
  final int id;
  final String name;
  final String username;
  final String email;
  final String phone;
  final String website;
  final String city;

  const Pengguna({
    required this.id,
    required this.name,
    required this.username,
    required this.email,
    required this.phone,
    required this.website,
    required this.city,
  });

  factory Pengguna.fromJson(Map<String, dynamic> json) {
    return Pengguna(
      id: json['id'] as int,
      name: json['name'] as String,
      username: json['username'] as String,
      email: json['email'] as String,
      phone: json['phone'] as String,
      website: json['website'] as String,
      city: json['address']['city'] as String,
    );
  }
}
```

```

Future<List<Pegguna>> ambilPegguna() async {
  final uri = Uri.parse('https://jsonplaceholder.typicode.com/users');
  final response = await http.get(uri).timeout(const Duration(seconds: 10));
  if (response.statusCode != 200) {
    throw Exception('Gagal memuat data (kode ${response.statusCode})');
  }
  final List<dynamic> data = jsonDecode(response.body);
  return data
    .map((e) => Pegguna.fromJson(e as Map<String, dynamic>))
    .toList();
}

class PeggunaPage extends StatefulWidget {
  const PeggunaPage({super.key});

  @override
  State<PeggunaPage> createState() => _PeggunaPageState();
}

class _PeggunaPageState extends State<PeggunaPage> {
  late Future<List<Pegguna>> _future;

  @override
  void initState() {
    super.initState();
    _future = ambilPegguna();
  }

  void _muatUlang() {
    setState(() {
      _future = ambilPegguna();
    });
  }

  // Latihan 2: dipakai oleh RefreshIndicator
  Future<void> _tarikMuatUlang() async {
    _muatUlang();
    try {
      await _future;
    } catch (_) {
      // galat ditampilkan oleh FutureBuilder
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        // Latihan 4: jumlah pengguna setelah data berhasil dimuat
        title: FutureBuilder<List<Pegguna>>(
          future: _future,
          builder: (context, snapshot) {
            final jumlah = snapshot.hasData ? ' (${snapshot.data!.length})' : '';
            return Text('Daftar Pegguna$jumlah');
          },
        ),
      actions: [
        IconButton(icon: const Icon(Icons.refresh), onPressed: _muatUlang),
      ],
    ),
    body: FutureBuilder<List<Pegguna>>(
      future: _future,
      builder: (context, snapshot) {
        if (snapshot.connectionState == ConnectionState.waiting) {
          return const Center(child: CircularProgressIndicator());
        }
        if (snapshot.hasError) {
          return Center(
            child: Padding(
              padding: const EdgeInsets.all(24),
              child: Column(
                mainAxisAlignment: MainAxisAlignment.min,
                children: [

```

```

        const Icon(Icons.error_outline, size: 48, color: Colors.red),
        const SizedBox(height: 8),
        Text(
          'Terjadi kesalahan:\n${snapshot.error}',
          textAlign: TextAlign.center,
        ),
        const SizedBox(height: 12),
        ElevatedButton(
          onPressed: _muatUlang,
          child: const Text('Coba lagi'),
        ),
      ),
    ],
  ),
);
}
final data = snapshot.data!;
return RefreshIndicator(
  onRefresh: _tarikMuatUlang,
  child: data.isEmpty
    ? ListView(
        children: const [
          SizedBox(height: 200),
          Center(child: Text('Tidak ada data')),
        ],
      )
    : ListView.builder(
        itemCount: data.length,
        itemBuilder: (context, i) {
          final p = data[i];
          return ListTile(
            leading: CircleAvatar(child: Text(p.name[0])),
            title: Text(p.name),
            subtitle: Text(p.email),
            trailing: const Icon(Icons.chevron_right),
            onTap: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (_) => DetailPeggunaPage(pegguna: p),
                ),
              );
            },
          );
        },
      ),
);
}
}
}

class DetailPeggunaPage extends StatelessWidget {
  final Pegguna pegguna;

  const DetailPeggunaPage({super.key, required this.pegguna});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(pegguna.name)),
      body: ListView(
        children: [
          ListTile(
            leading: const Icon(Icons.person),
            title: Text(pegguna.username),
          ),
          ListTile(
            leading: const Icon(Icons.email),
            title: Text(pegguna.email),
          ),
          ListTile(
            leading: const Icon(Icons.phone),

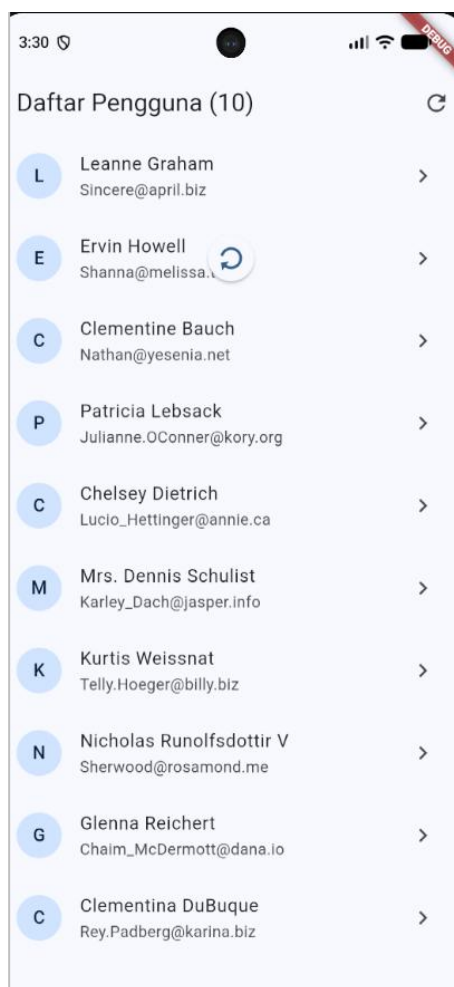
```

```

        title: Text(pengguna.phone),
      ),
      ListTile(
        leading: const Icon(Icons.language),
        title: Text(pengguna.website),
      ),
      ListTile(
        leading: const Icon(Icons.location_city),
        title: Text(pengguna.city),
      ),
    ],
  ),
);
}
}

```

Pada gambar berikut, judul AppBar sudah menampilkan jumlah pengguna (10) dan indikator tarik-untuk-muat-ulang tampak di tengah daftar.



Gambar 4. Hasil Latihan Mandiri: jumlah pengguna di AppBar dan RefreshIndicator

5. Tugas: Aplikasi Daftar Postingan

Aplikasi Daftar Postingan memakai endpoint `/posts` dan `/posts/{id}/comments`. Kode dipisah menjadi tiga bagian dalam `main.dart`: model (Post dan Komentar dengan `fromJson`), layanan pengambil data (`PostService`), dan UI (`PostPage`, `DetailPostPage`, dan `ErrorView`). Halaman

utama menampilkan judul dan potongan isi postingan; mengetuk postingan membuka halaman detail berisi isi lengkap dan daftar komentar yang diambil dari Future kedua. Kedua halaman menangani status loading dan galat dengan tombol Coba lagi.

Kode 4. main.dart untuk Tugas Daftar Postingan

```
import 'dart:convert';

import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MyApp());

// ===== MODEL =====

class Post {
  final int id;
  final int userId;
  final String title;
  final String body;

  const Post({
    required this.id,
    required this.userId,
    required this.title,
    required this.body,
  });

  factory Post.fromJson(Map<String, dynamic> json) {
    return Post(
      id: json['id'] as int,
      userId: json['userId'] as int,
      title: json['title'] as String,
      body: json['body'] as String,
    );
  }
}

class Komentar {
  final int id;
  final String name;
  final String email;
  final String body;

  const Komentar({
    required this.id,
    required this.name,
    required this.email,
    required this.body,
  });

  factory Komentar.fromJson(Map<String, dynamic> json) {
    return Komentar(
      id: json['id'] as int,
      name: json['name'] as String,
      email: json['email'] as String,
      body: json['body'] as String,
    );
  }
}

// ===== LAYANAN (PENGAMBIL DATA) =====

class PostService {
  static const _base = 'https://jsonplaceholder.typicode.com';

  static Future<List<dynamic>> _getList(String path) async {
```

```

        final response = await http
            .get(Uri.parse('$_base$path'))
            .timeout(const Duration(seconds: 10));
        if (response.statusCode != 200) {
            throw Exception('Gagal memuat data (kode ${response.statusCode})');
        }
        return jsonDecode(response.body) as List<dynamic>;
    }

    static Future<List<Post>> ambilPosts() async {
        final data = await _getList('/posts');
        return data.map((e) => Post.fromJson(e as Map<String, dynamic>)).toList();
    }

    static Future<List<Komentar>> ambilKomentar(int postId) async {
        final data = await _getList('/posts/$postId/comments');
        return data
            .map((e) => Komentar.fromJson(e as Map<String, dynamic>))
            .toList();
    }
}

// ===== UI =====

class MyApp extends StatelessWidget {
    const MyApp({super.key});

    @override
    Widget build(BuildContext context) {
        return MaterialApp(
            title: 'Daftar Postingan',
            theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
            home: const PostPage(),
        );
    }
}

class ErrorView extends StatelessWidget {
    final Object? error;
    final VoidCallback onRetry;

    const ErrorView({super.key, required this.error, required this.onRetry});

    @override
    Widget build(BuildContext context) {
        return Center(
            child: Padding(
                padding: const EdgeInsets.all(24),
                child: Column(
                    mainAxisAlignment: MainAxisAlignment.min,
                    children: [
                        const Icon(Icons.error_outline, size: 48, color: Colors.red),
                        const SizedBox(height: 8),
                        Text(
                            'Terjadi kesalahan:\n$error',
                            textAlign: TextAlign.center,
                        ),
                        const SizedBox(height: 12),
                        ElevatedButton(onPressed: onRetry, child: const Text('Coba lagi')),
                    ],
                ),
            ),
        );
    }
}

// ----- Halaman daftar postingan -----

class PostPage extends StatefulWidget {
    const PostPage({super.key});

```

```

@override
State<PostPage> createState() => _PostPageState();
}

class _PostPageState extends State<PostPage> {
  late Future<List<Post>> _future;

  @override
  void initState() {
    super.initState();
    _future = PostService.ambilPosts();
  }

  void _muatUlang() {
    setState(() {
      future = PostService.ambilPosts();
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Daftar Postingan'),
        actions: [
          IconButton(icon: const Icon(Icons.refresh), onPressed: _muatUlang),
        ],
      ),
      body: FutureBuilder<List<Post>>(
        future: _future,
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          }
          if (snapshot.hasError) {
            return ErrorView(error: snapshot.error, onRetry: _muatUlang);
          }
          final data = snapshot.data!;
          return ListView.builder(
            itemCount: data.length,
            itemBuilder: (context, i) {
              final p = data[i];
              return ListTile(
                title: Text(p.title, maxLines: 1, overflow: TextOverflow.ellipsis),
                subtitle: Text(
                  p.body.replaceAll('\n', ' '),
                  maxLines: 2,
                  overflow: TextOverflow.ellipsis,
                ),
                trailing: const Icon(Icons.chevron_right),
                onTap: () {
                  Navigator.push(
                    context,
                    MaterialPageRoute(
                      builder: (_) => DetailPostPage(post: p),
                    ),
                  );
                },
              );
            },
          );
        },
      ),
    );
  }
}

// ----- Halaman detail + komentar (Future kedua) -----

class DetailPostPage extends StatefulWidget {
  final Post post;

  const DetailPostPage({super.key, required this.post});
}

```

```

    @override
    State<DetailPostPage> createState() => _DetailPostPageState();
}

class _DetailPostPageState extends State<DetailPostPage> {
    late Future<List<Komentar>>> _future;

    @override
    void initState() {
        super.initState();
        _future = PostService.ambilKomentar(widget.post.id);
    }

    void _muatUlang() {
        setState(() {
            _future = PostService.ambilKomentar(widget.post.id);
        });
    }

    @override
    Widget build(BuildContext context) {
        final post = widget.post;
        return Scaffold(
            appBar: AppBar(title: Text('Postingan #${post.id}')),
            body: ListView(
                padding: const EdgeInsets.all(16),
                children: [
                    Text(post.title, style: Theme.of(context).textTheme.titleLarge),
                    const SizedBox(height: 8),
                    Text(post.body),
                    const Divider(height: 32),
                    Text('Komentar', style: Theme.of(context).textTheme.titleMedium),
                    const SizedBox(height: 8),
                    FutureBuilder<List<Komentar>>>(
                        future: _future,
                        builder: (context, snapshot) {
                            if (snapshot.connectionState == ConnectionState.waiting) {
                                return const Padding(
                                    padding: EdgeInsets.all(24),
                                    child: Center(child: CircularProgressIndicator()),
                                );
                            }
                            if (snapshot.hasError) {
                                return ErrorView(error: snapshot.error, onRetry: _muatUlang);
                            }
                            final komentar = snapshot.data!;
                            if (komentar.isEmpty) {
                                return const Text('Belum ada komentar');
                            }
                            return Column(
                                children: komentar
                                    .map(
                                        (k) => Card(
                                            child: ListTile(
                                                title: Text(k.name),
                                                subtitle: Text('${k.email}\n${k.body}'),
                                                isThreeLine: true,
                                            ),
                                        ),
                                    )
                                    .toList(),
                            );
                        },
                    ),
                ],
            ),
        );
    }
}

```



Gambar 5. Halaman Daftar Postingan

D. Jawaban Pertanyaan Refleksi

1. Apa yang terjadi bila Future dibuat di dalam build() dan bukan di initState()?

Setiap kali layar dibangun ulang (misalnya setelah setState, rotasi layar, atau kembali dari halaman lain), permintaan HTTP dikirim lagi. Akibatnya data dimuat ulang terus-menerus, tampilan kembali ke loading, dan kuota terbuang. initState() hanya dipanggil sekali sehingga Future dibuat satu kali.

2. Apa perbedaan snapshot.hasError dengan memeriksa response.statusCode?

snapshot.hasError menangkap exception, misalnya jaringan terputus, timeout, atau galat penguraian JSON. response.statusCode memeriksa respons yang berhasil diterima server tetapi menandakan kegagalan (404, 500). Paket http tidak melempar exception untuk kode

tersebut, sehingga status harus diperiksa dan dilempar sebagai Exception agar sampai ke `snapshot.error`. Keduanya diperlukan agar semua jenis kegagalan tertangani.

3. Mengapa data JSON sebaiknya diubah menjadi class model?

Class model memberi tipe data yang jelas, mendukung autocomplete, dan membuat kesalahan (kunci salah atau nilai null) terdeteksi saat penguraian, bukan tersebar di UI. Pemetaan JSON juga terpusat di `fromJson` sehingga kode lebih rapi dan mudah dirawat dibanding memakai Map secara langsung.

4. Mengapa UI wajib menyediakan status loading dan error?

Pengambilan data butuh waktu dan bisa gagal. Tanpa status loading, pengguna mengira aplikasi macet; tanpa status error, layar kosong atau aplikasi bermasalah tanpa penjelasan dan tanpa cara untuk mencoba lagi.

E. Kesimpulan

Melalui praktikum ini, pengambilan data dari REST API di Flutter berhasil dilakukan menggunakan paket `http`, `Future`, dan `FutureBuilder`. Data JSON diubah menjadi objek model lewat `fromJson`, lalu ditampilkan dengan penanganan tiga keadaan (loading, galat, dan data) beserta tombol coba lagi. Latihan mandiri menambah data bersarang, tarik-untuk-muat-ulang, penanganan daftar kosong, dan jumlah data di AppBar, sedangkan tugas Daftar Postingan menerapkan pemisahan model, layanan, dan UI dengan dua `Future` pada dua halaman.